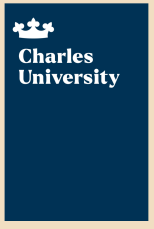
Fundamental limits of proofs of programs

Cultures of Programming — Part 5

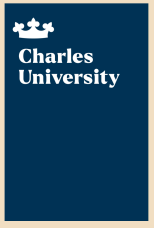
Tomas Petricek, Charles University, Prague

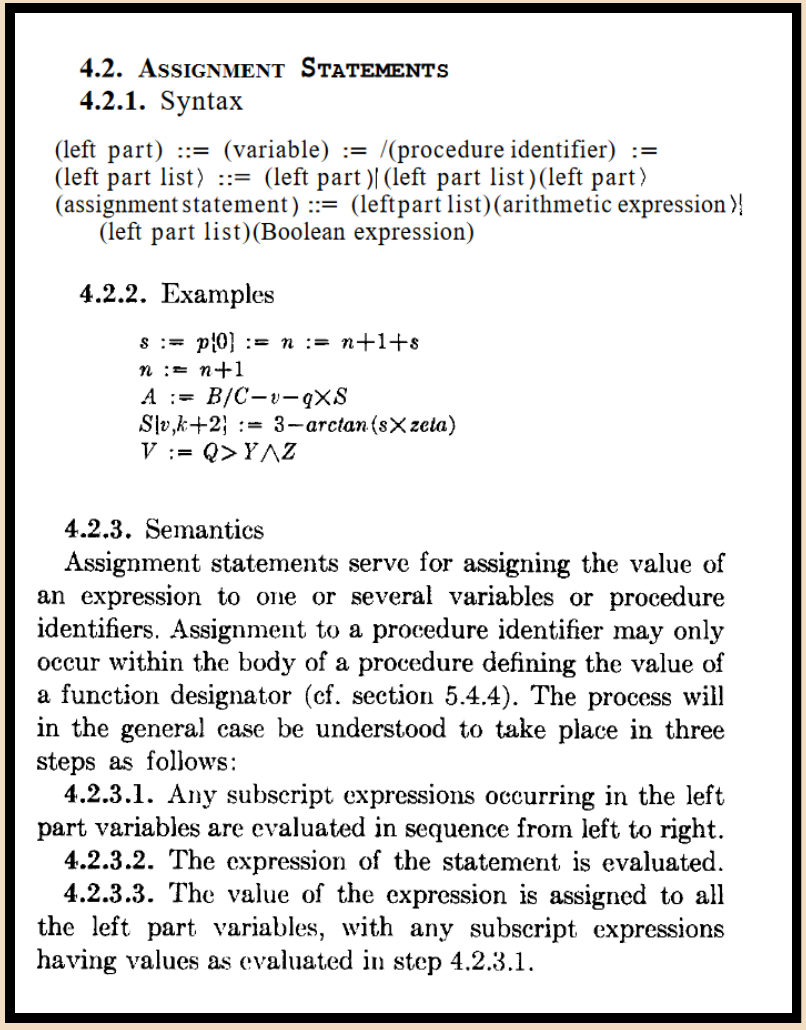
tomas@tomasp.net
https://tomasp.net
@tomasp.net



Algol

Object of mathematical beauty





4.2. Assignment Statements

4.2.1. Syntax

⟨left part⟩ ::= ⟨variable⟩ := | ⟨procedure identifier⟩ :=
⟨left part list⟩ ::= ⟨left part⟩ | ⟨left part list⟩⟨left part⟩
⟨assignment statement⟩ ::= ⟨left part list⟩⟨arithmetic expression⟩ |
⟨left part list⟩⟨Boolean expression⟩

4.2.2. Examples

$s := p[0] := n := n+1+s$
$n := n+1$
$A := B/C-v-q\times S$
$S[v,k+2] := 3-arctan(s\times zeta)$
$V := Q>Y\wedge Z$

4.2.3. Semantics

Assignment statements serve for assigning the value of an expression to one or several variables or procedure identifiers. Assignment to a procedure identifier may only occur within the body of a procedure defining the value of a function designator (cf. section 5.4.4). The process will in the general case be understood to take place in three steps as follows:

4.2.3.1. Any subscript expressions occurring in the left part variables are evaluated in sequence from left to right.

4.2.3.2. The expression of the statement is evaluated.

4.2.3.3. The value of the expression is assigned to all the left part variables, with any subscript expressions having values as evaluated in step 4.2.3.1.

Algol report (1960)

Formally describes language syntax

What about semantics?

What happens when programs are executed?

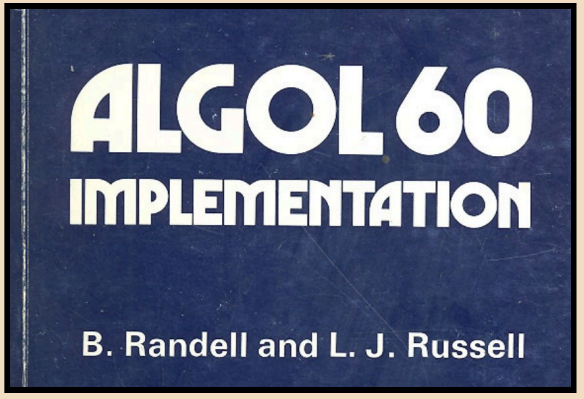
Turns out to be harder!

Charles University

Early semantics of Algol

Microalgol (McCarthy, 1964)

- Small subset, but includes `goto`
- Defined as a function $\texttt{micro}(\pi, \xi)$
- Mathematically-defined interpreter



Axiomatic basis (Hoare, 1969)

- Axioms of fortran constructs
- For example: $\{p(f)\}x := f\{p(x)\}$.
- Admits that `goto` is difficult

Charles University

5. The Semantics of Microalgol

```
                value (τ , ξ) = if isvar (τ) then  c( τ,ξ)
else if isconst (τ) then val (τ)
else if issum (τ) then value (addend (τ), ξ) + value (augend (τ), ξ)
else if isdiff (τ) then value (subtrahend (τ), ξ)  - value (minuend (τ), ξ)
else if isprod (τ) then value (multiplier (τ), ξ) x value (multiplicand (τ), ξ)
else if isquotient (τ) then value (numerator (τ), ξ)/value (denominator(τ),ξ)
else if iscond (τ) then (if value (proposition (τ), ξ) then
              value(antecedent (τ), ξ) else value (consequent (τ), ξ))
else if isequal (τ) then (value(lefteq (τ), ξ) = value (righteq (τ), ξ))
else if isless (τ) then (value (leftl (τ), ξ) < value (rightl(τ), ξ))
```

We now can write

```
micro (π ,ξ) = (λ n .if end (π ,n ) then ξ
else (λ s . if assignment (s) then
micro(π, a(sn, n + 1, a(left(s), value (right(s),ξ),ξ)))
                else if goto (s) then
micro (π, a(sn, if value (proposition (s), ξ) then
                numb (destination (s),π) else n + 1, ξ)))
(statement (n,π)) (c (sn,ξ))
```

Microalgol definition by McCarthy

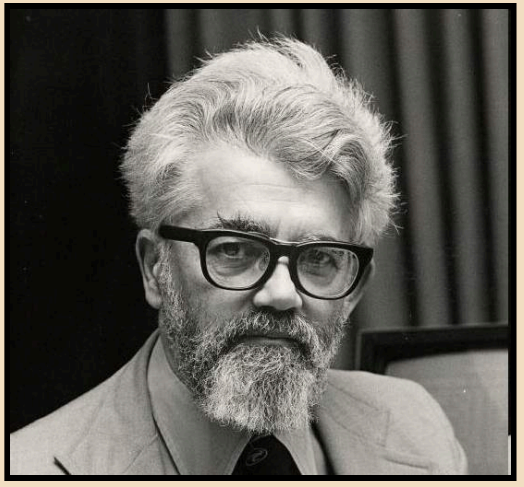
Program π, execution state ξ

Charles University

Early days optimism

"Those difficulties can be resolved and that a clear description of the state of an Algol computation will clarify the problem of compiler design."

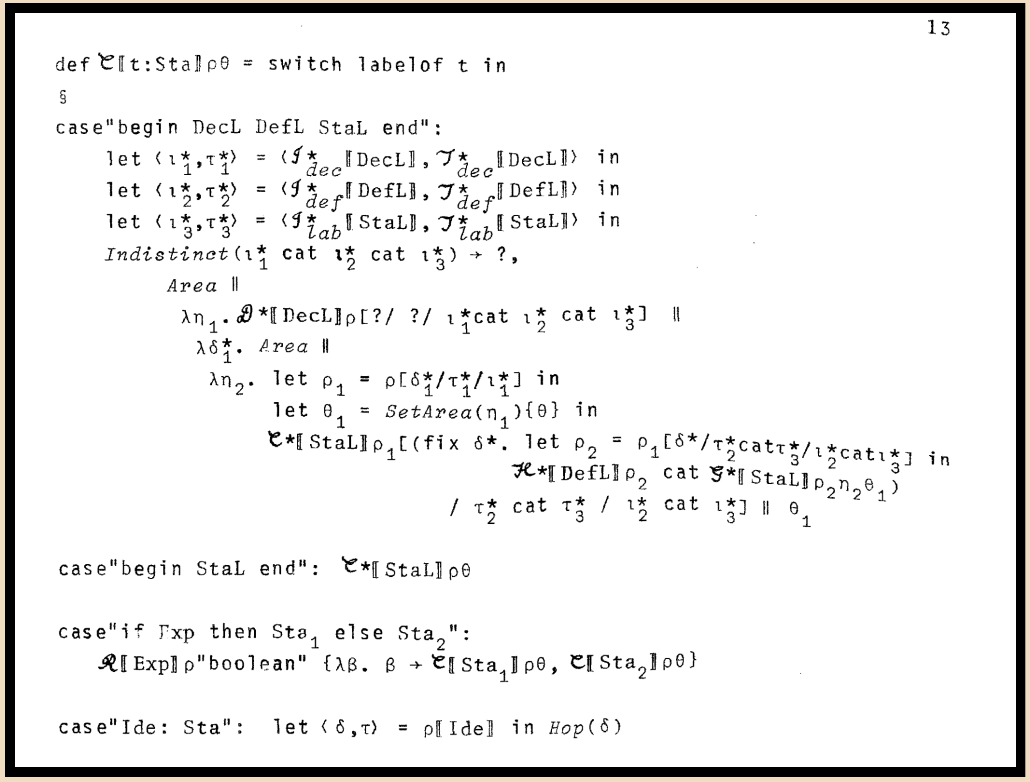
(McCarthy, 1964)



"The practical advantages of program proving will eventually outweigh the difficulties."

(Hoare, 1969)





```
                                                                    13

def 𝔈⟦t:Sta⟧ρθ = switch labelof t in
§
case"begin DecL DefL StaL end":
    let ⟨ι*1,τ*1⟩ = ⟨ℐ*dec⟦DecL⟧, 𝒯*dec⟦DecL⟧⟩ in
    let ⟨ι*2,τ*2⟩ = ⟨ℐ*def⟦DefL⟧, 𝒯*def⟦DefL⟧⟩ in
    let ⟨ι*3,τ*3⟩ = ⟨ℐ*lab⟦StaL⟧, 𝒯*lab⟦StaL⟧⟩ in
    Indistinct(ι*1 cat ι*2 cat ι*3) → ?,
        Area ‖
         λη1.𝒟*⟦DecL⟧ρ[?/ ?/ ι*1cat ι*2 cat ι*3] ‖
          λδ*1. Area ‖
           λη2. let ρ1 = ρ[δ*1/τ*1/ι*1] in
                let θ1 = SetArea(η1){θ} in
                𝔈*⟦StaL⟧ρ1[(fix δ*. let ρ2 = ρ1[δ*/τ*2catτ*3/ι*2catι*3] in
                                    ℜ*⟦DefL⟧ρ2 cat 𝔊*⟦StaL⟧ρ2η2θ1)
                          / τ*2 cat τ*3 / ι*2 cat ι*3] ‖ θ1

case"begin StaL end": 𝔈*⟦StaL⟧ρθ

case"if Exp then Sta1 else Sta2":
   𝔄⟦Exp⟧ρ"boolean" {λβ. β → 𝔈⟦Sta1⟧ρθ, 𝔈⟦Sta2⟧ρθ}

case"Ide: Sta": let ⟨δ,τ⟩ = ρ⟦Ide⟧ in Hop(δ)
```

Mosses (1974)

Method by Strachey

Assign mathematical object to constructs

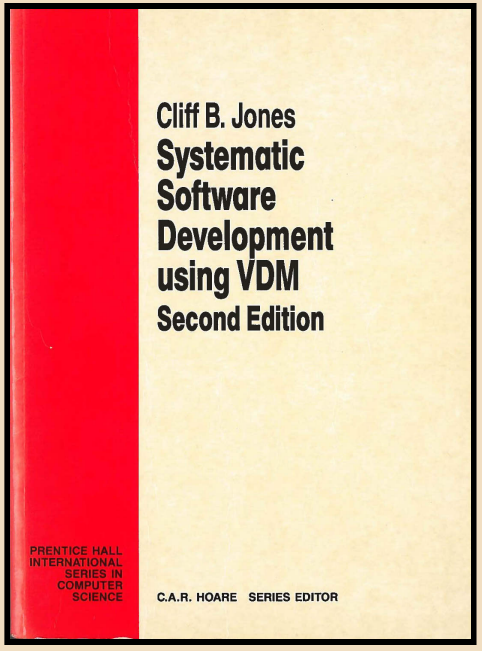
Terms of λ calculus!

Charles University

Vienna Development Method

Tool-supported definition

- Aim to model more complex PL/1
- Vienna Definition Language (VDL)
- Algol as the starting point
- 50 page long definition



Formalising Algol in VDL

- Machine-checked definition
- 67 page definition (1968) ala McCarthy
- book-chapter length (1978) after Mosses

Charles University

Algol

Many formal definitions

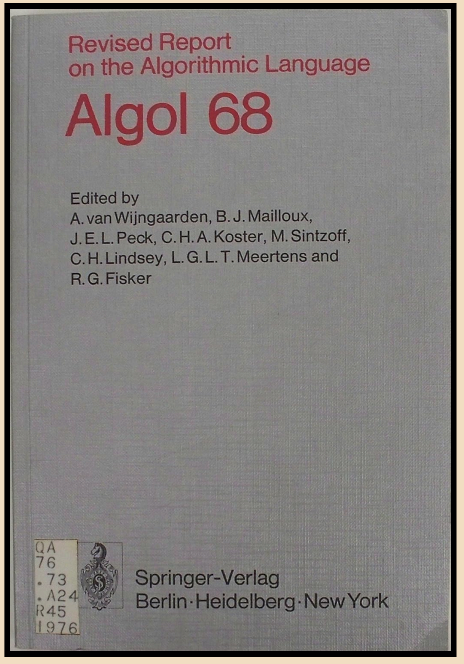
- Small subset vs. full language
- Surprised "is's as long as it is"!
- "Algol 60 is just not nearly as well-behaved"
- 60 pages of definitions, no theorems!



From Algol 60 to Algol 68

Further development

- Universal language aims
- More orthogonal definitions



Sources of complexity

- Using meta-grammar and grammar
- Modes (`ref`,`proc`,`struct`,`union`,`flex`)
- Coercions (dereferencing, deproceduring, uniting, widening, rowing, and voiding)

Maybe not so bad today!

Charles University

Mathematical programme

Programming problems



"Facing tasks of completely different
and still growing scope and scale."

From "expressing programs in a language"
to "language to support good practices."

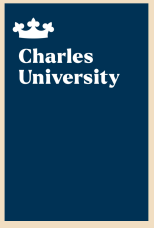
Responses to complications

- IFIP WG 2.3 on Programming Methodology
- NATO Software Engineering conference (1968)

Charles University

Proofs of programs

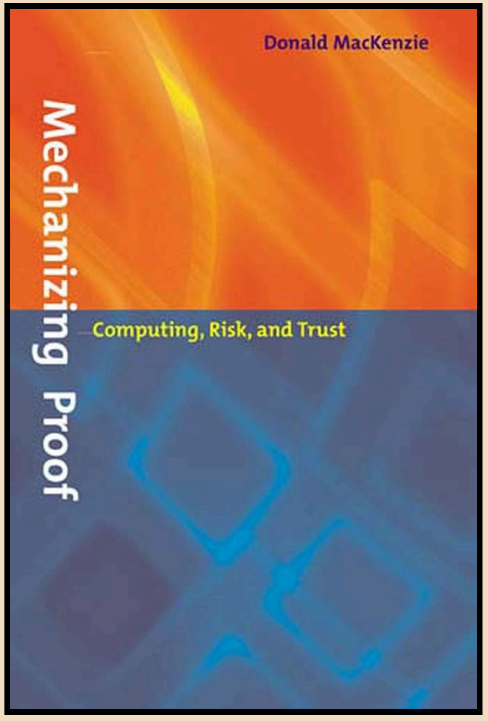
Social processes



De Millo, Lipton & Perlis

Met at Yale University seminar

- De Millo visiting Lipton
- Presenting verification research
- Extensive algorithm proof

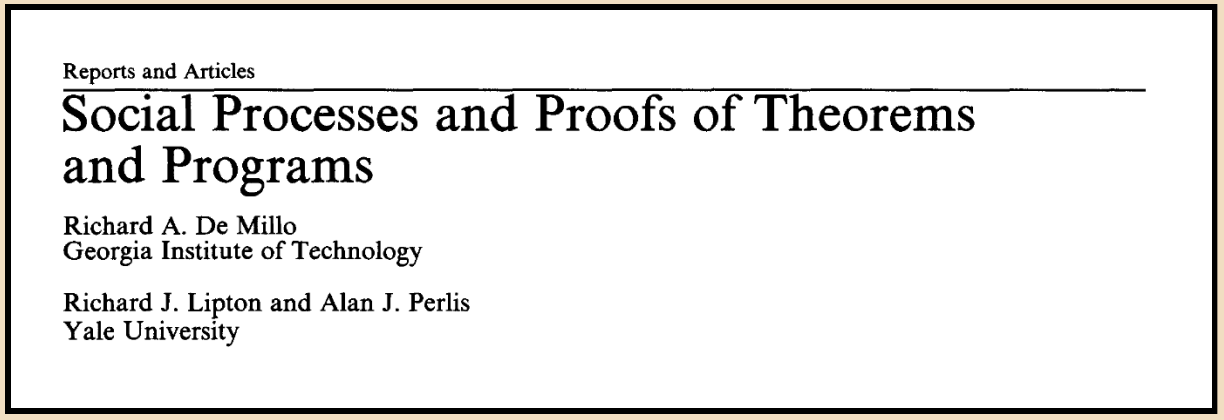


Perlis' question

"Why does it take 20 pages to prove a program that is obviously correct?"

What does it take to convince a human?

Charles University



Reports and Articles

Social Processes and Proofs of Theorems and Programs

Richard A. De Millo
Georgia Institute of Technology

Richard J. Lipton and Alan J. Perlis
Yale University

Proofs in logic

Finite sequences of propositions, each of which is an axiom, an assumption or follows from earlier propositions by a rule of inference.

Proofs in practice

Mix of natural and formal language, aimed to convince another qualified colleague

Charles University

Source of confidence

Central to mathematical practice

- Taught at universities
- Discussed at whiteboard sessions
- Mathematicians enjoy elegance & tricks
- Believe it could be elaborated



Proofs of programs differ

- Bound to be secondary
- Programs are written to be run
- Certification does not play the same role
- Repetitive, large number of cases

Charles University

Letters to the editor

Computer scientists

- A pamphlet from the Middle Ages (Dijkstra)



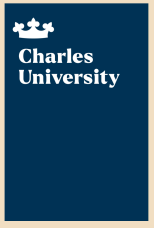
Industry readership

- Wisdom, fairness, style, rigour, and wit
- First article enjoyed since joining the ACM
- Cannot recall a single instance in which a proof of a program's correctness would have been useful

Charles University

Proofs of programs

Engineering and managerial answers



Proofs of programs

- **De Millo et al. mix many cultures**
  Mathematical with humanistic and engineering twist
- **Purely mathematical verification bound to fail**
  Real-world software systems are too complex
- **1960s proofs by hand no longer suffice**
  Response comes from different perspectives!
- **Managerial and engineering approaches**
  Scaling proof methods to real-world systems

Charles University



QUALITY ASSURANCE

Cleanroom Software Engineering

Harlan D. Mills, *Information Systems Institute*
Michael Dyer *and* **Richard C. Linger**, *IBM Federal Systems Division*

Software quality can be engineered under statistical quality control and delivered with better quality. The Cleanroom process gives management an engineering approach to release reliable products.

Recent experience demonstrates that software can be engineered under statistical quality control and that certified reliability statistics can be provided with delivered software. IBM's Cleanroom process[1] has uncovered a surprising synergy between mathematical verification and statistical testing of software, as well as a major difference between mathematical fallibility and debugging fallibility in people.

With the Cleanroom process, you can engineer software under statistical quality control. As with cleanroom hardware development, the process's first priority is defect prevention rather than defect removal (of course, any defects not prevented should be removed). This first priority is achieved by using human mathematical verification in place of program debugging to prepare software for system test.

Its next priority is to provide valid, statistical certification of the software's quality through representative-user testing at the system level. The measure of quality is the mean time to failure in appropriate units of time (real or processor time) of the delivered product. The certification takes into account the growth of reliability achieved during system testing before delivery.

To gain the benefits of quality control during development, Cleanroom software engineering requires a development cycle of concurrent fabrication and certification of product increments that accumulate into the system to be delivered. This lets the fabrication process be altered on the basis of early certification results to achieve the quality desired.

Cleanroom experience

Typical of our experience with the Cleanroom process were three projects: an IBM language product (40,000 lines of code), an Air Force contract helicopter flight program (35,000 lines), and a NASA contract space-transportation planning system (45,000 lines). A major finding in these cases was that human verification, even though fallible, could replace debugging in software development — even informal human verification can produce

0740-7459/87/0900/0019/$01.00 ©1987 IEEE

Harlan Mills at IBM
PhD in game theory

Extension of Chief Programmer Teams

"Software engineering without mathematical verification is no more than a buzzword."

Charles University

Cleanroom Software Engineering

Surgical team model

- Extends Chief Programmer Teams
- Lead surgeon, supported by assistants
- Controlled environment (cleanroom)

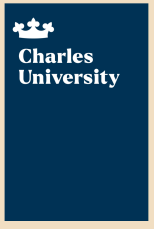


Managing reliability

- Review conformance to specification rigorously
- Degree of rigor a managerial choice
- Statistical control of *reliability over a probability distribution of usage scenarios*

Charles University

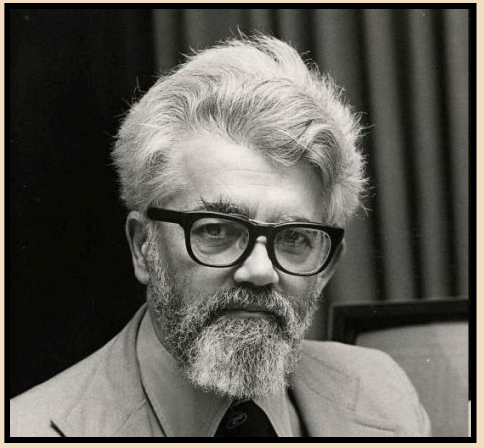
Mechanical proof checking



Mechanical proof checking

"One should prove that [a program] meets its specifications, and this proof should be checked by a computer program."

(McCarthy, 1963)



"A mechanical theorem prover does not act as an oracle ... but as an implacable sceptic that insists on all assumptions being stated and all claims justified."

(Rushby & Von Henke, 1990)

Mechanical proof checking

- **Logic theorist** (Newell & Simon, 1950-1960s)
  AI will be able to do more than just check!
- **Robert Boyer & J Strother Moore** (1970s)
  Proves (with luck) properties of LISP-like programs
- **LCF prover** (Milner, 1970s)
  Motivated by compiler proofs, ML meta-langauge
- **Separation logic** (2000s)
  Infer static analyser by Facebook

Charles University

Formal verification in practice

sel4 microkernel
A range of mission critical projects

CompCert compiler
Faster than gcc without (untrusted) optimizations

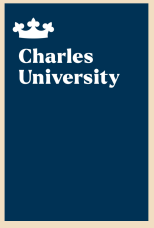
Verification tools
Static Driver Verifier at Microsoft, Infer at Facebook

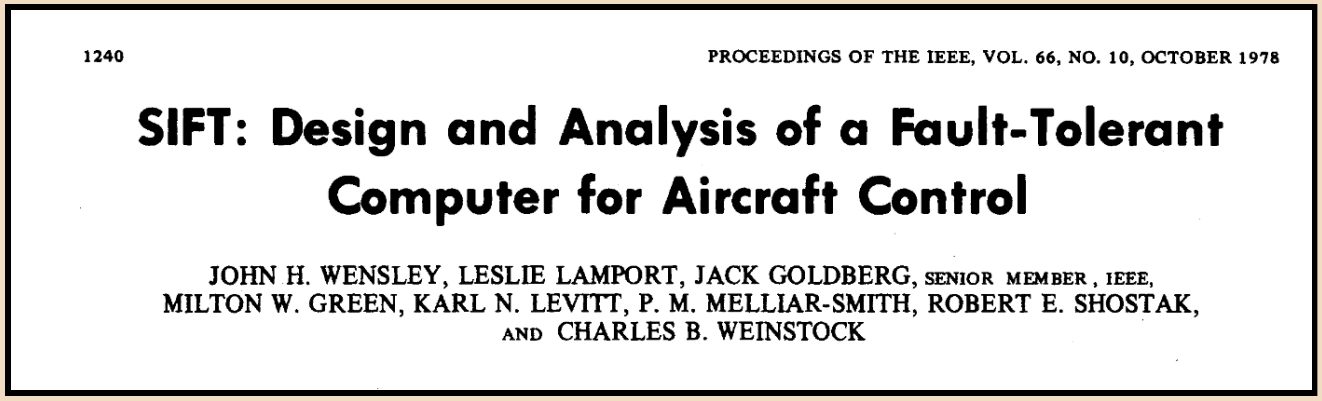


Charles University

Proofs of programs

Fundamental limitations





1240 PROCEEDINGS OF THE IEEE, VOL. 66, NO. 10, OCTOBER 1978

SIFT: Design and Analysis of a Fault-Tolerant Computer for Aircraft Control

JOHN H. WENSLEY, LESLIE LAMPORT, JACK GOLDBERG, SENIOR MEMBER, IEEE, MILTON W. GREEN, KARL N. LEVITT, P. M. MELLIAR-SMITH, ROBERT E. SHOSTAK, AND CHARLES B. WEINSTOCK

Software Implemented Fault Tolerance

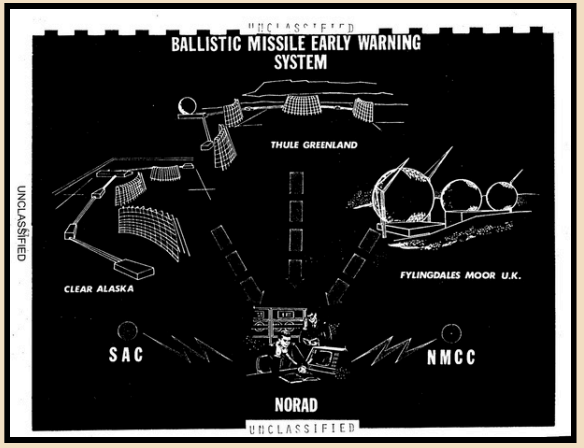
Proof could only be completed if they also had a precise specification of how the underlying microprocessor executes instructions and also the timing of such execution.

Boyer and Moore (1983)

Charles University

Ballistic Missile Early-Warning System

Brian Cantwell Smith (1985)



- Mistook rising moon for a missile
- If it was verified, should it be trusted?

Inherent limitations

- Models ignore timing or the moon
- Programs are not abstract mathematical entities

Proofs check compatibility of two descriptions!

Hardware verification

VIPER microprocessor

The first commercial microprocessor with both a formal specification and a proof that the chip conforms to it



Avra Cohn's comment

Device can be described … and the description verified; but …there is no way to assure the accuracy of the description

Lawsuit about the nature of the proof!

Charles University

Proofs

Programs vs mathematics

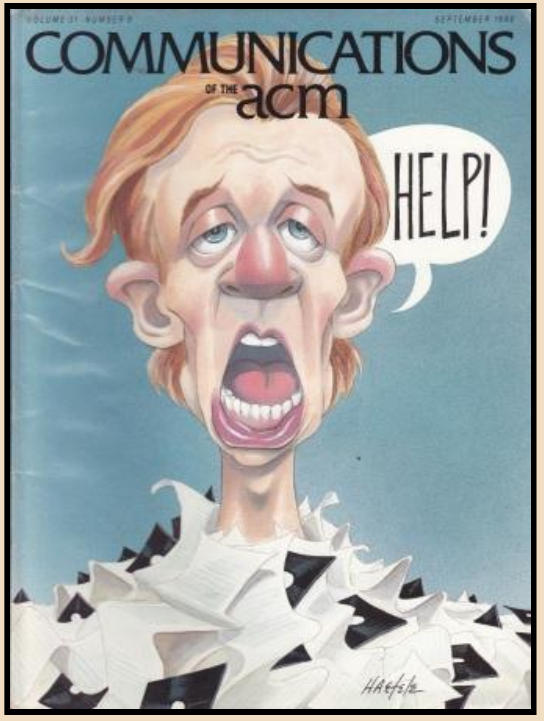
- ≠ Programs are not mathematical entities
-  Real hardware and real world!
-  Non-determinism, temperature, etc.
-  Mathematical model is never fully detailed

Charles University

The very idea (Fetzer, 1987)

Verification is nonsensical

- Program is a different kind of thing than mathematics
- Category mistake



What can verification mean?

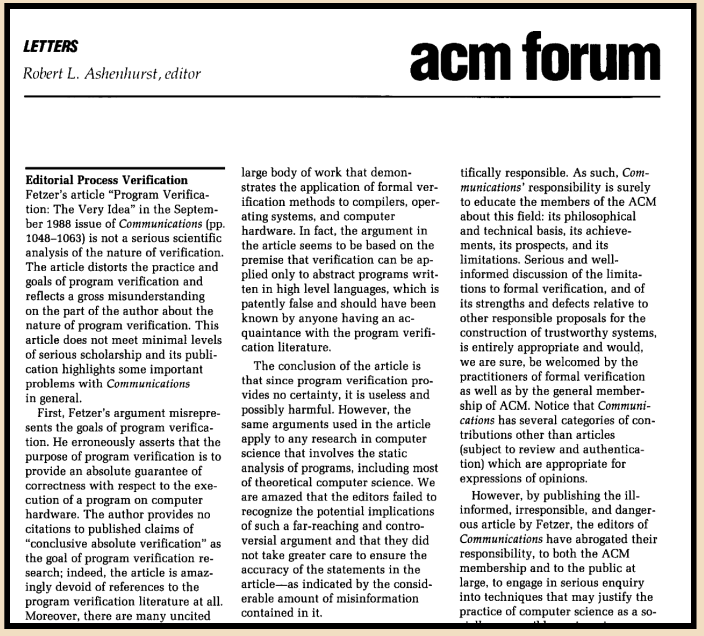
- **Machine as formal entity**
  Valid, but does not say anything
- **Machine as empirical description**
  Proof is empirical knowledge

Charles University

Reaction to the paper

Letter by computer scientists

- Attacks a parody of verification
- Not a serious scientific analysis
- Ill-informed, irresponsible, and dangerous article



LETTERS

Robert L. Ashenhurst, editor

acm forum

Editorial Process Verification

Fetzer's article "Program Verification: The Very Idea" in the September 1988 issue of *Communications* (pp. 1048–1063) is not a serious scientific analysis of the nature of verification. The article distorts the practice and goals of program verification and reflects a gross misunderstanding on the part of the author about the nature of program verification. This article does not meet minimal levels of serious scholarship and its publication highlights some important problems with *Communications* in general.

First, Fetzer's argument misrepresents the goals of program verification. He erroneously asserts that the purpose of program verification is to provide an absolute guarantee of correctness with respect to the execution of a program on computer hardware. The author provides no citations to published claims of "conclusive absolute verification" as the goal of program verification research; indeed, the article is amazingly devoid of references to the program verification literature at all. Moreover, there are many uncited large body of work that demonstrates the application of formal verification methods to compilers, operating systems, and computer hardware. In fact, the argument in the article seems to be based on the premise that verification can be applied only to abstract programs written in high level languages, which is patently false and should have been known by anyone having an acquaintance with the program verification literature.

The conclusion of the article is that since program verification provides no certainty, it is useless and possibly harmful. However, the same arguments used in the article apply to any research in computer science that involves the static analysis of programs, including most of theoretical computer science. We are amazed that the editors failed to recognize the potential implications of such a far-reaching and controversial argument and that they did not take greater care to ensure the accuracy of the statements in the article—as indicated by the considerable amount of misinformation contained in it.

tifically responsible. As such, *Communications*' responsibility is surely to educate the members of the ACM about this field: its philosophical and technical basis, its achievements, its prospects, and its limitations. Serious and well-informed discussion of the limitations to formal verification, and of its strengths and defects relative to other responsible proposals for the construction of trustworthy systems, is entirely appropriate and would, we are sure, be welcomed by the practitioners of formal verification as well as by the general membership of ACM. Notice that *Communications* has several categories of contributions other than articles (subject to review and authentication) which are appropriate for expressions of opinions.

However, by publishing the ill-informed, irresponsible, and dangerous article by Fetzer, the editors of *Communications* have abrogated their responsibility, to both the ACM membership and to the public at large, to engage in serious enquiry into techniques that may justify the practice of computer science as a so-

Reveals something true

- Earlier critiques say similar thing!
- Ignored or hidden in long reports…

Charles University

Beyond mathematical culture



Old-school proofs on whiteboard

Do not scale for problems like Algol definition!



Response involves other cultures

Managerial Cleanroom, engineering tools



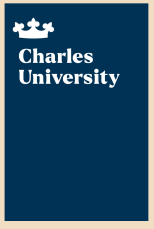
Shifting meaning of proofs

From social activity to checkmark with caveats



Different kinds of proofs

CompCert vs. programming language theory



Cultures of Programming



Home → Continue to Part 6

Charles University